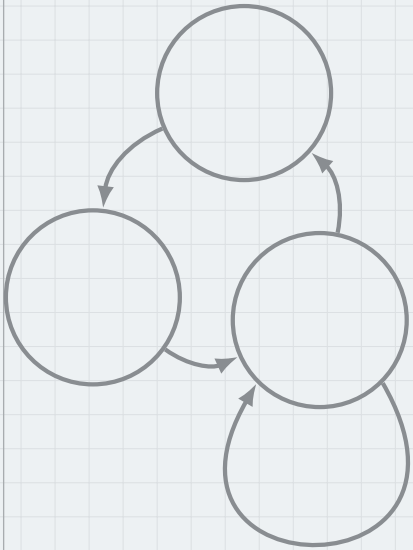


FINITE STATE MACHINES IN EMBEDDED C

Architectural Patterns for STM32 & Microcontrollers

Nerio Andrés Montoya
namontoy@unal.edu.co
Universidad Nacional de Colombia · 2026-S2

Agenda

Core Concepts

From Diagram to C

Table-Driven FSM

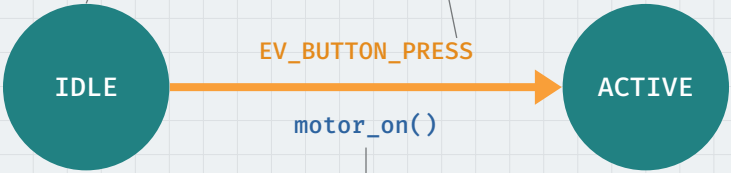
Choosing and Pitfalls

Core Concepts

State, Event, Transition

STATE (Teal): The system condition at a given time. Embodies what the system knows.

EVENT (Orange): The trigger (Interrupt, Timer, Sensor input).



TRANSITION & ACTION (Blue): The execution output and movement to the next state.

Moore vs. Mealy

Moore Paradigm

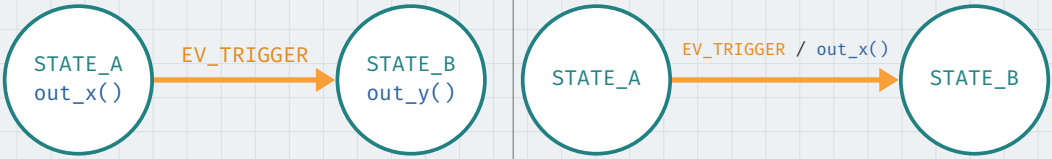
`Output = f(Current State)`

Output is bound to the state itself.

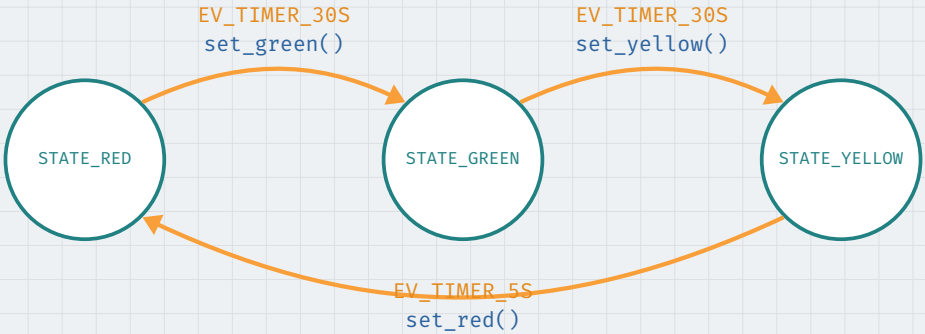
Mealy Paradigm

`Output = f(Current State + Input Event)`

Output is bound to the transition.



Worked Example: A Traffic Light



Current State	Input Event	Next State	Output Action
STATE_RED	EV_TIMER_30S	STATE_GREEN	set_green()
STATE_GREEN	EV_TIMER_30S	STATE_YELLOW	set_yellow()
STATE_YELLOW	EV_TIMER_5S	STATE_RED	set_red()

From Diagram to C

Step 1: Mapping Vocabulary to Memory

```
typedef enum {
   STATE_IDLE,
   STATE_ACTIVE,
   STATE_WAIT,
   STATE_ERROR,
   NUM_STATES
} State_t;
```

`typedef enum` enforces type safety and prevents arbitrary integer assignment. The compiler will warn if an invalid state is passed.

```
typedef enum {
   EV_NONE,
   EV_BUTTON_PRESS,
   EV_TIMEOUT,
   EV_ERROR,
   NUM_EVENTS
} Event_t;
```

Events must be explicitly defined. A continuous loop polls for these events. `NUM_STATES` and `NUM_EVENTS` are sentinels: they size the tables and enable range checks.

Pattern 1: The Switch-Case Architecture

```
State_t current_state = STATE_IDLE;
Event_t current_event;

int main(void) {
    hardware_init();

    while(1) {
        current_event = get_next_event();

        switch(current_state) {
            // State logic executes here
        }
    }
}
```

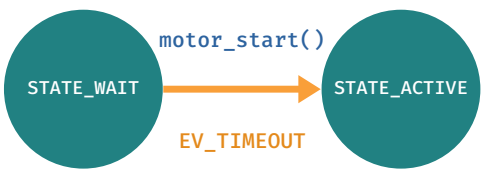
The Infinite Loop: Firmware never exits. It constantly spins, evaluates events, and reacts.



Non-Blocking Rule: No `delay()` functions are allowed here. Blocking halts the entire state machine.

Step 2: Mapping FSM Nodes to C Switch-Case Logic

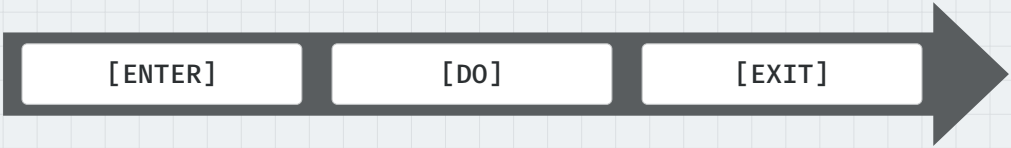
FSM DIAGRAM VISUALIZATION



C CODE IMPLEMENTATION

```
case STATE_WAIT:
    if (current_event == EV_TIMEOUT) {
        motor_start();
        current_state = STATE_ACTIVE;
    }
    break;
```

Managing State Actions: Entry, Do, Exit



Runs exactly once upon arrival

Spins continuously while in state

Runs exactly once before leaving

```
case STATE_ACTIVE:
  if (state_entry_flag) {
    enable_sensors();          // [ENTRY] Action
    state_entry_flag = 0;
  }

  read_sensor_data();          // [DO] Action

  if (current_event == EV_ERROR) {
    disable_sensors();         // [EXIT] Action
    current_state = STATE_ERROR;
    state_entry_flag = 1;
  }
  break;
```

Use **flags** or **discrete sub-states** to ensure Entry and Exit actions only fire once, while the **Do** action spins freely in the non-blocking **while(1)** loop.

Table-Driven FSM

Pattern 2: The Table-Driven FSM

	EV_NONE	EV_BUTTON_PRESS	EV_TIMEOUT	EV_ERROR
STATE_IDLE	STATE_IDLE	STATE_ACTIVE	STATE_IDLE	STATE_ERROR
STATE_ACTIVE	STATE_ACTIVE	STATE_ACTIVE	STATE_WAIT	STATE_ERROR
STATE_WAIT	STATE_WAIT	STATE_IDLE	STATE_WAIT	STATE_ERROR
STATE_ERROR	STATE_ERROR	STATE_ERROR	STATE_ERROR	STATE_ERROR

```
const State_t transition_table[NUM_STATES][NUM_EVENTS] = {
    // EV_NONE        EV_BUTTON_PRESS  EV_TIMEOUT     EV_ERROR
    {  STATE_IDLE,    STATE_ACTIVE,    STATE_IDLE,    STATE_ERROR },  // STATE_IDLE
    {  STATE_ACTIVE,  STATE_ACTIVE,    STATE_WAIT,    STATE_ERROR },  // STATE_ACTIVE
    {  STATE_WAIT,    STATE_IDLE,      STATE_WAIT,    STATE_ERROR },  // STATE_WAIT
    {  STATE_ERROR,   STATE_ERROR,     STATE_ERROR,   STATE_ERROR }   // STATE_ERROR
};
```

Moving execution logic from code space into memory space (ROM). Essential for highly complex state machines to avoid massive switch-case spaghetti.

The O(1) Execution Engine

```
int main(void) {
  while(1) {
    current_event = get_next_event();

    // The entire state machine transition logic:
    if (current_event != EV_NONE) {
      if ((current_state < NUM_STATES) &&
          (current_event < NUM_EVENTS)) {
        current_state = transition_table
          [current_state][current_event];
      } else {
        current_state = STATE_ERROR; // recover
      }
    }
  }
}
```

- **Execution Speed:** Extremely fast. Constant-time $O(1)$ array lookup.
- **Constraint:** Requires a dense matrix. You must explicitly define behavior for all invalid state/event combinations (e.g., gracefully ignoring them by mathematically mapping back to the current state).
- **Safeguard:** Validate both indices before the lookup; out-of-range values recover to `STATE_ERROR`.

Choosing and Pitfalls

Architectural Comparison Matrix

Pattern	Complexity Level	Memory Footprint	Execution Speed	Best Use Case
Switch-Case	Low to Medium	Low (Code space)	O(N) evaluation	Simple UI menus, basic motor control, < 10 states.
Table-Driven	Medium	High (Dense 2D Array in ROM)	O(1) Constant Time	Protocol parsers, complex state routing, predictable timing.

Firmware Traps & Hardware Realities



Catch the Cosmic Rays: Always use a '`default:`' case in your `switch` statements. Unhandled memory states caused by **stack corruption** or **cosmic rays** must be caught and safely **reset**.



Beware the ISR (Interrupt Service Routine): If **state variables** or **event flags** are modified inside a hardware interrupt, they must be declared as 'volatile'. Otherwise, compiler optimization will break the FSM.



Separate the What from the How: Partition code strictly. The FSM should dictate WHAT happens (the policy). Separate driver functions should handle HOW it happens (the physical hardware mechanism).

FSM Patterns and the Power of 10

JPL Power of 10 rule	In our FSM code
1. Simple control flow: no `goto`, `setjmp`, or recursion	States are flat `case` labels; a transition is an assignment to the state variable, never a call back into the FSM.
2. Fixed loop bounds	The main loop is the one intentional non-terminating loop; each pass handles at most one event and one transition.
3. No dynamic memory after initialization	States and events are enums; the transition table is `const` data in ROM.
4. Short functions (about 60 lines)	As states multiply, give each state its own short handler and keep the `switch` as a dispatcher.
5. Assertions with explicit recovery	Range-check state and event before the lookup, and keep a `default:` case: both recover to `STATE_ERROR`.
6. Smallest possible scope	Keep `current_state` `static` inside the FSM module and expose it only through functions.
9. Restricted pointers (no function pointers)	Switch-case and table indexing need none, which is why a function-pointer FSM is not part of this course.

G. J. Holzmann, "The Power of 10: Rules for Developing Safety-Critical Code," *IEEE Computer*, June 2006 (NASA/JPL).

FSM Meets the ISR

```
volatile Event_t pending_event = EV_NONE;

void EXTI0_IRQHandler(void) {
    pending_event = EV_BUTTON_PRESS;
    // clear the EXTI pending flag
}

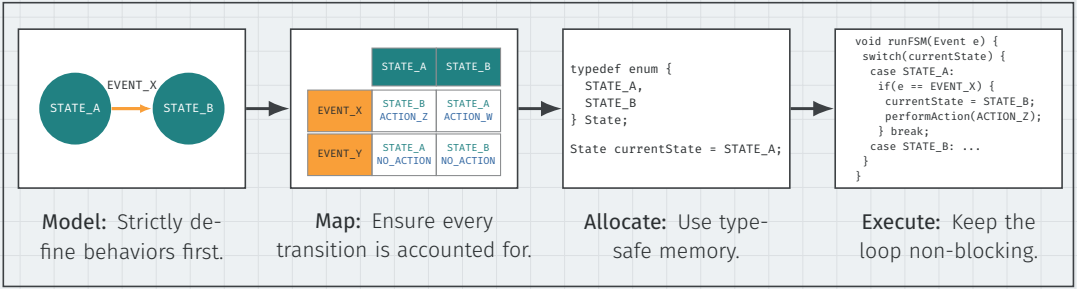
Event_t get_next_event(void) {
    Event_t ev;
    __disable_irq();
    ev = pending_event;
    pending_event = EV_NONE;
    __enable_irq();
    return ev;
}
```

volatile: the flag changes outside the normal program flow, so the compiler must re-read it every time.

The ISR: record the event and return. No FSM logic in here.

Critical section: read-and-clear must not be interrupted, or an event could be lost. The main loop's FSM then consumes the event.

State Machine Implementation Process



Model: Strictly define behaviors first.

Map: Ensure every transition is accounted for.

Allocate: Use type-safe memory.

Execute: Keep the loop non-blocking.

The state machine separates robust firmware from fragile spaghetti code. Model rigorously before coding, and let the structure of your code mirror the geometry of your design.